DSC211_module_8

November 26, 2024

**

Module 8: Exploratory Data Analysis (Deep Dive)

**

This module provides a comprehensive introduction to exploratory data analysis (EDA) techniques, essential for understanding and preparing datasets for further analysis. You will be able to learn various methods for conducting sanity checks, univariate analysis, bivariate analysis, missing value treatment, outlier treatment, and handling categorical data types. At the end of the module, you are expected to develop necessary skills for carrying out exploratory data anslysis using real world data.

Content

Sanity Checks, Univariate Analysis, Bivariate Analysis, Missing value treatment, Outlier Treatment; handling categorical data types

Learning Outcomes

By the end of this module the learner should be able to:

1. Identify the key techniques used in exploratory data analysis.
2. Explain the importance of EDA and how it contributes to data preparation for further analysis
3. Demonstrate the ability to perform exploratory data analysis on real-world datasets by applying appropriate EDA echniques

**

Sanity checks

** Sanity check is a process in the exploratory data analysis that ensures the data is valid and reliable. In any data science project you will carry out, it is important to ensure that the data is reliable and consistent. There are various ways for ensuring data consistency;

1. Unique values: This ensures that the columns that are required to be unique are unique
2. Handling duplicates
3. Ensuring consistency in the labels of categorical data. For example using “female” verses “f”, “F”. There should be consistency in the values used for all the categorical columns.

You are going to see how to use this methods using the black friday dataset, which is cointains customer transactions As a normal practice, we will start by loading the dataset as follows;

[3]:
```
import pandas as pd
data=pd.read_csv("C:/Users/User/Dropbox/OUK Documentts/Python for data science module development/Datasets/black_friday.csv")
data.head()
```

[3]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category_1	Product_Category_2	Product_Category_3	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	NaN	NaN	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	6.0	14.0	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	NaN	NaN	1422
3	1000001	P00085442	F	0-17	10	A	2	0	12	14.0	NaN	1057
4	1000002	P00285442	M	55+	16	C	4+	0	8	NaN	NaN	7969

Lets check to see if the User_ID column has duplicated valuesd

[6]:
```
# Check for duplicate IDs
duplicate_ids = data[data.duplicated(subset='User_ID', keep=False)]

duplicate_ids.head()
```

[6]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category_1	Product_Category_2	Product_Category_3	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	NaN	NaN	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	6.0	14.0	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	NaN	NaN	1422
3	1000001	P00085442	F	0-17	10	A	2	0	12	14.0	NaN	1057
4	1000002	P00285442	M	55+	16	C	4+	0	8	NaN	NaN	7969

```
0                NaN                NaN       8370
1                6.0               14.0      15200
2                NaN                NaN       1422
3               14.0                NaN       1057
4                NaN                NaN       7969
```

Because this data cointains the transactions, it is possible to have the same customer purchasing the different items at different times.We can check if the whole transactions are duplicated

[9]:
```
duplicate_records = data[data.duplicated(keep=False)]
duplicate_records.shape
```

[9]:
```
(2, 12)
```

[11]:
```
data.shape
```

[11]:
```
(550069, 12)
```

As we can see we have two records that are duplicated. In this case we can drop the duplicate as follows

[14]:
```
df_cleaned = data.drop_duplicates(keep='first')
```

We can then check the new dataseet to see if there are still duplicates in it

[17]:
```
df_cleaned.shape
```

[17]:
```
(550068, 12)
```

As we can see from the shape attributes results our rows have reduced by one. Not using the keep=‘first’ attributes will delete all the records that are duplicated

Ensuring consistency in the categorical variables naming

Lets check if our categorical column “City_Category” has consistent values. We can do this by checking the unique values in our column as follows

[21]:
```
data["City_Category"].unique()
```

[21]:
```
array(['A', 'C', 'B', 'b'], dtype=object)
```

From the results, we can see that the column has four unique values, with B and b which is a representation of the same thing. We can handle this by replacing the values that have “b” to “B” using the replace method as follows

[24]:
```
data['City_Category'] = data['City_Category'].replace('b', 'B')
```

After replacing we can now check to see if our column has only three unique values as follows

[27]:
```
data['City_Category'] .unique()
```

[27]:
```
array(['A', 'C', 'B'], dtype=object)
```

Data integrity test

In addition to checking the consistency of the data, you can also check the integrity of the data by

1. Indetifying missing data and handling it
2. Detecting the outliers
3. Ensuring that the numerical data fall into expected ranges. For example if the dataset cointains the records of students marks you do not expect the values for the marks to go beyond 100 and below 0

In the earlier sections, we had talked about handling the missing values using the isna() function, dropna() and fillna() functions. In this section we are going to look at detecting outliers and ensuring consistent range in a dataset

Detecting outliers

Outliers are values in our dataset that do not lie at the normal range of the data. The most easy way to detect is to use a box plot. In this section we are using the student_performance dataset.

[30]:
```
import pandas as pd
data=pd.read_csv("C:/Users/User/Dropbox/OUK Documentts/Python for data science␣
 ↪module development/Datasets/Student_Performance.csv")
data.head()
```

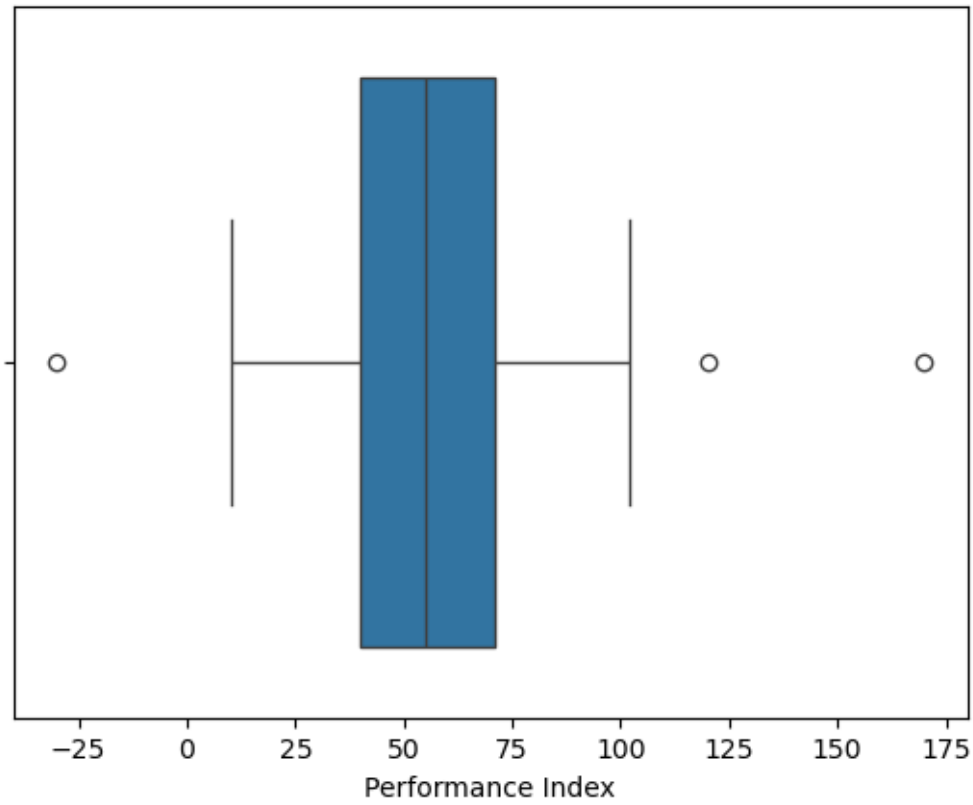
[30]:

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	\
0	7	99	Yes	9	
1	4	82	No	4	
2	8	51	Yes	7	
3	5	52	Yes	5	
4	7	75	No	8	

	Sample Question Papers Practiced	Performance Index
0	1	91
1	2	65
2	2	45
3	2	36
4	5	66

Lets check if the Performance Index column has an outlier by plotting a boxplot of it

[33]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data=pd.read_csv("C:/Users/User/Dropbox/OUK Documentts/Python for data science␣
 ↪module development/Datasets/Student_Performance.csv")
sns.boxplot(x=data['Performance Index'])
plt.show()
```



From the boxplot, we can see we have two outliers on the right and one outlier on the left. We can remove the outliers by filtering our dataset. We can set the set the maximum value to be 100 and the minimum value to be 0

[35]:
```
import pandas as pd
data=pd.read_csv("C:/Users/User/Dropbox/OUK Documentts/Python for data science␣
 ↪module development/Datasets/Student_Performance.csv")
data_new=data.loc[(data['Performance Index'] <= 100) & (data['Performance␣
 ↪Index'] > 0)]
print("Original dataset shape",data.shape)
print("Filtered dataset shape",data_new.shape)
```

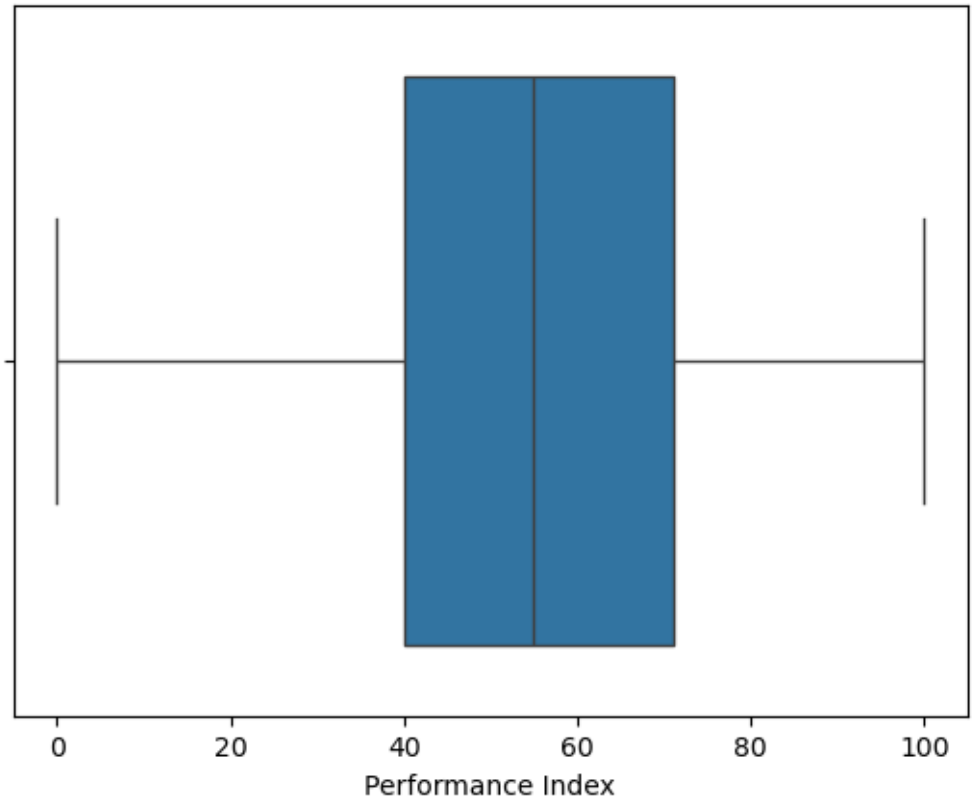
```
Original dataset shape (10003, 6)
Filtered dataset shape (9999, 6)
```

From the results we can see that our dataset has reduced by 4 elements

The other way of handling outliers is by replacing the outlier values with the a specified maximum or minimum

[54]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
data=pd.read_csv("C:/Users/User/Dropbox/OUK Documentts/Python for data science
 ↪module development/Datasets/Student_Performance.csv")
data.loc[data['Performance Index'] > 100] = 100
data.loc[data['Performance Index']<0 ] = 0
sns.boxplot(x=data['Performance Index'])
plt.show()
```



By replacing the outliers, we can see from our boxplot the outliers have been removed

Data Accuracy Checks

1. Cross-Field Validation: Cross validation ensures logical consistency between the variables in the dataset. For example, if you have a column of the birth date, it should be consistent with the age column.

[44]:
```
import pandas as pd
from datetime import datetime

# Sample DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Birth Date': ['2000-01-01', '1985-05-20', '1990-12-15'],
```

```
    'Age': [24, 39, 33]  # Assuming current year is 2024
}

df = pd.DataFrame(data)

# Convert 'Birth Date' to datetime
df['Birth Date'] = pd.to_datetime(df['Birth Date'])

# Calculate current year and compare with 'Age'
df['Calculated Age'] = datetime.now().year - df['Birth Date'].dt.year

# Check if 'Age' matches 'Calculated Age'
df['Age Matches'] = df['Age'] == df['Calculated Age']

print(df)
```

```
      Name Birth Date  Age  Calculated Age  Age Matches
0    Alice 2000-01-01   24              24         True
1      Bob 1985-05-20   39              39         True
2  Charlie 1990-12-15   33              34        False
```

Once you have noted the inconsistencies you can work on correcting them

Logical Consistency: Logical consistency ensures that there is Valid logical relationships between variable (e.g., start date should be before end date).

[48]:
```
import pandas as pd

# Sample DataFrame
data = {
    'Task': ['Task 1', 'Task 2', 'Task 3', 'Task 4'],
    'Start Date': ['2024-01-10', '2024-02-01', '2024-03-15', '2024-04-01'],
    'End Date': ['2024-01-15', '2024-01-30', '2024-03-10', '2024-04-05']
}

df = pd.DataFrame(data)

# Convert 'Start Date' and 'End Date' to datetime format
df['Start Date'] = pd.to_datetime(df['Start Date'])
df['End Date'] = pd.to_datetime(df['End Date'])

# Display the original DataFrame
print("Original DataFrame:")
print(df)
# Validate that 'Start Date' is before 'End Date'
df['Valid Dates'] = df['Start Date'] < df['End Date']

# Display the DataFrame with the validation results
```

```
print("\nDataFrame with Validation:")
print(df)
# Filter rows where 'Start Date' is not before 'End Date'
inconsistent_dates = df[df['Valid Dates'] == False]

# Display tasks with inconsistent dates
print("\nTasks with Inconsistent Dates:")
print(inconsistent_dates)
```

```
Original DataFrame:
     Task Start Date   End Date
0  Task 1 2024-01-10 2024-01-15
1  Task 2 2024-02-01 2024-01-30
2  Task 3 2024-03-15 2024-03-10
3  Task 4 2024-04-01 2024-04-05

DataFrame with Validation:
     Task Start Date   End Date  Valid Dates
0  Task 1 2024-01-10 2024-01-15         True
1  Task 2 2024-02-01 2024-01-30        False
2  Task 3 2024-03-15 2024-03-10        False
3  Task 4 2024-04-01 2024-04-05         True

Tasks with Inconsistent Dates:
     Task Start Date   End Date  Valid Dates
1  Task 2 2024-02-01 2024-01-30        False
2  Task 3 2024-03-15 2024-03-10        False
```

3. Aggregation Consistency: Sum or aggregate data to ensure that totals match expected value

Distribution Checks

Distribution checks are used to check the distribution of the variables

Summary Statistics: Summary statistics give us the idea about our data.Summary statistics include the mean, the median, the mode, min, max, and the standard deviation. In pandas, we can get the summary statistics using the pandas describe method. The mean tells us the avarage values in the columns, median tells as the median value and standard deviation tells us how disperse the values are.

If we run the describe() method on our data we can see that we get summary

[56]:
```
data.describe()
```

[56]:
```
       Hours Studied  Previous Scores   Sleep Hours  \
count   10003.000000     10003.000000  10003.000000
mean        5.020694        69.447466      6.557833
std         3.067766        17.362454      2.344958
min         0.000000         0.000000      0.000000
25%         3.000000        54.000000      5.000000
```

```
50%          5.000000        69.000000      7.000000
75%          7.000000        85.000000      8.000000
max        100.000000       100.000000    100.000000

       Sample Question Papers Practiced  Performance Index
count                      10003.000000       10003.000000
mean                           4.611816          55.231830
std                            3.309075          19.233049
min                            0.000000           0.000000
25%                            2.000000          40.000000
50%                            5.000000          55.000000
75%                            7.000000          71.000000
max                          100.000000         100.000000
```

We can also calculate the individual statistics such as the mean, standard deviation, the median each of the columns. Lets say we want to see how spread is the previous scores in our dataset. We can do that using the standard deviation as follows

[59]:
```
data["Previous Scores"].std()
```

[59]:
```
17.36245441638453
```

We can also calculate the mean as follows

[61]:
```
data["Previous Scores"].mean()
```

[61]:
```
69.44746576027192
```

By looking at the stadard deviation value, we can conclude that most of the students in this data had the previous score as 69.44746576027192 +/-17.36245441638453. This mean that the scores lie between 52 and 87. This information tells us how spread our data is.

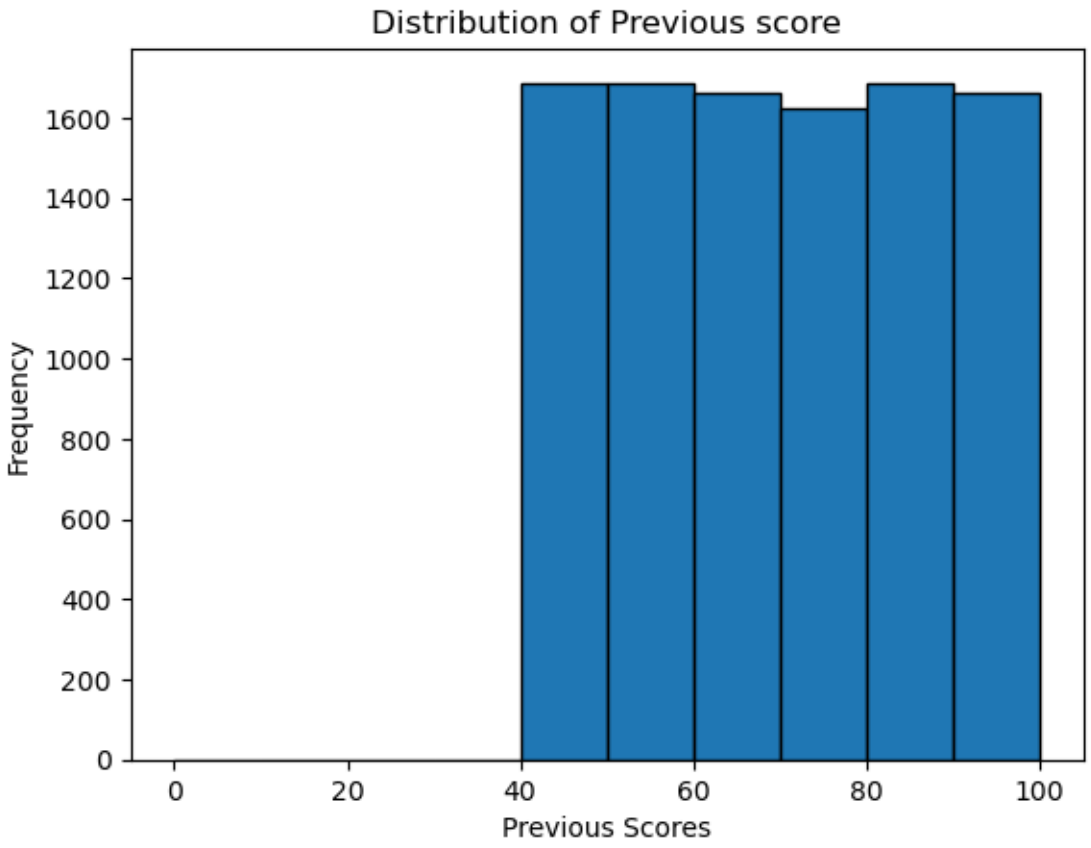
0.1 Histograms:

We can also use an histogram to tell us how our data is distrubted graphically. Using the same example for the previous score, we can create an histogram as follows

[66]:
```
# Plotting the histogram for the 'Age' column
plt.hist(data['Previous Scores'], bins=10, edgecolor='black')

# Adding labels and title
plt.title("Distribution of Previous score")
plt.xlabel("Previous Scores")
plt.ylabel("Frequency")

# Display the plot
plt.show()
```



We can see from the histogram, that we do not have any scores below 40. This helps you in understading your data better

Correlation Analysis:

In addition to doing the summary statistics and the histograms, we can also check correlations between variables to identify unexpected relationships or collinearity in our dataset. This will help us in indetfying which features to include in our models. If features are highly correleted, then you might need to drop some of them as they might not add much to the model performance.

Pandas provides a built in method called "corr" for quickly finding out the correletion of the variables in your dataset.

[75]:
```
#Select columns for correlation analysis
data_corr=data[["Hours Studied","Previous Scores", "Sleep Hours"]]
#Perform correlation analysis
data_corr.corr()
```

[75]:

	Hours Studied	Previous Scores	Sleep Hours
Hours Studied	1.000000	0.006537	0.371323
Previous Scores	0.006537	1.000000	0.026428
Sleep Hours	0.371323	0.026428	1.000000

From the results, we can see that the hours studied, the previous score and the sleep hours are not highly correleted

Sampling Bias Checks

Sample Representativeness: Ensure that the sample data is representative of the population (e.g., no over-representation of a particular group).

One of the major challenges experienced with machine learning algorithms is the fact that sometimes they are biased. The major reason why machine learning algorithms have bias in them is because of the bias that is inherent in the dataset. So when preparing your data, you might need to consider looking if your data is representative enough of the poplation.

You can check for samlpling bias by using statistical test such as the chi-square and the t-test against the known population known values. To do the test, we can use the python stats library.

[ ]: